

Programação Orientada a Objetos

Introdução a orientação a Objetos, conhecendo os Paradigmas de programação.

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



1 O que são paradigmas de programação?

O que são Paradigmas de Programação

Paradigmas de programação são **modelos conceituais** que definem:

- Como os programas são estruturados;
- Como os dados são representados;
- Como as instruções são executadas.

Eles influenciam diretamente:

- A forma de pensar soluções computacionais;
- A organização e manutenção do código;
- A escolha das linguagens de programação.

Paradigma Imperativo

No paradigma imperativo, o programa é descrito como uma **sequência de comandos** que alteram o estado do sistema.

- Ênfase em **como** o problema é resolvido;
- Uso intenso de variáveis, atribuições e laços;
- Execução passo a passo.

Exemplos de linguagens: C, Pascal, Fortran.

Imperativo (estruturado)

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = []

for number in numbers:
    if (number % 2 == 0):
        even_numbers.append(number)

print(even_numbers)
```



Figura: Paradigma Imperativo

Paradigma Procedural

O paradigma procedural é uma extensão do paradigma imperativo, com foco em:

- Organização do código em **procedimentos ou funções**;
- Reutilização de código;
- Redução da complexidade por decomposição.

```
1 function add(x: number, y: number) {  
2   return x + y;  
3 }  
4  
5 const result = add(10, 20);  
6 console.log(result);  
7
```

Figura: Paradigma Procedural

Características principais:

- Separação entre dados e procedimentos;
- Estrutura modular.

Exemplos: C, Pascal.

Paradigma Procedural

O paradigma procedural também é muito utilizado nos hooks do Wordpress

```
hooks.php
1 <?php
2 /**
3  * Setup theme hooks
4  *
5  * @package Total
6  * @subpackage Framework
7  * @author Alexander Clarke
8  * @copyright Copyright (c) 2014, Symple Workz LLC
9  * @link http://www.wpexplorer.com
10 * @since Total 1.0.0
11 */
12
13 /**
14  * Main Header Hooks
15  *
16  * @since Total 1.0.0
17  */
18 function wpex_hook_header_before() {
19     do_action( 'wpex_hook_header_before' );
20 }
21 function wpex_hook_header_after() {
22     do_action( 'wpex_hook_header_after' );
23 }
24 function wpex_hook_header_top() {
25     do_action( 'wpex_hook_header_top' );
26 }
27 function wpex_hook_header_inner() {
28     do_action( 'wpex_hook_header_inner' );
29 }
30 function wpex_hook_header_bottom() {
31     do_action( 'wpex_hook_header_bottom' );
32 }
```

Figura: Hooks do wordpress

Paradigma Orientado a Objetos

O paradigma orientado a objetos (POO) organiza o software em **classes**, que são moldes para os **objetos**, entidades do mundo real.



Figura: Classe



Figura: Objeto

```
public class Cachorro {  
    String raça;  
    String cor;  
  
    void latir() {  
        // Código para o cachorro latir  
        System.out.println(x: "Au au!");  
    }  
  
    void correr() {  
        // Código para o cachorro correr  
        System.out.println(x: "O cachorro está correndo!");  
    }  
}
```

Figura: Paradigma Orientado a Objetos

Paradigma Orientado a Objetos

```
public class Cachorro {  
    String raca;  
    String cor;  
  
    void latir() {  
        // Código para o cachorro latir  
        System.out.println(x: "Au au!");  
    }  
  
    void correr() {  
        // Código para o cachorro correr  
        System.out.println(x: "O cachorro está correndo!");  
    }  
}
```

Figura: Paradigma Orientado a Objetos

Conceitos fundamentais:

- Classes e objetos;
- Encapsulamento;
- Herança;
- Polimorfismo.

Paradigma Orientado a Objetos

A POO favorece:

- Manutenibilidade;
- Reuso de código;
- Modelagem mais próxima da realidade.

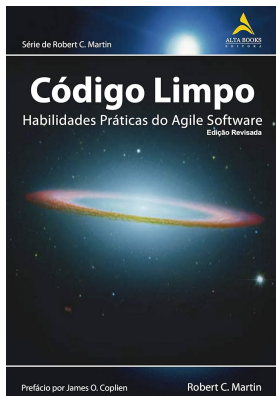


Figura: Clean Code

O paradigma funcional baseia-se em princípios da **matemática funcional**.

- Programas são construídos a partir de **funções**;
- Evita estados mutáveis e efeitos colaterais;
- Ênfase em **imutabilidade** dos dados;
- Uso de funções puras e recursão.

Esse paradigma favorece código previsível e mais fácil de testar.

- Funções como cidadãos de primeira classe;
- Alto nível de abstração;
- Facilita paralelismo e concorrência.

Linguagens associadas:

- Haskell;
- Lisp;
- Scala;
- Uso parcial em Java, Python e JavaScript.

Exemplo — JavaScript no Paradigma Funcional

No paradigma funcional, evita-se modificar estados e utiliza-se funções puras e funções de alta ordem.

```
const temperaturas = [28, 30, 26, 32];
```

```
// Função pura
```

```
const celsiusParaFahrenheit = c =>  
  (c * 9/5) + 32;
```

```
// Funções de alta ordem
```

```
const fahrenheit = temperaturas  
  .map(celsiusParaFahrenheit)  
  .filter(t => t > 86);
```

```
console.log(fahrenheit);
```

Exemplo de uso de imutabilidade, map e filter.



No paradigma lógico, o programa é definido por:

- **Fatos;**
- **Regras;**
- Consultas lógicas.

O programador descreve o **problema**, e o sistema encontra a solução por meio de inferência lógica.

Exemplo — Prolog no Paradigma Lógico

No paradigma lógico, o programa é composto por **fatos** e **regras**, e a execução ocorre por meio de **consultas**.

% Fatos

pai(joao, maria).

pai(joao, jose).

mae(ana, maria).

mae(ana, jose).

% Regra

progenitor(X, Y) :-

 pai(X, Y);

 mae(X, Y).

% Consulta (exemplo)

% ?- progenitor(joao, maria).

O sistema utiliza inferência lógica para responder às consultas.



- Baseado em lógica de predicados;
- Separação entre conhecimento e controle;
- Uso de mecanismos de inferência automática.

Linguagem principal:

- Prolog.

É muito utilizado em sistemas especialistas e inteligência artificial.

No paradigma declarativo, o foco está em **o que deve ser feito**, não em **como fazer**.

- O programador descreve o resultado desejado;
- A execução fica a cargo do sistema;
- Menor controle explícito do fluxo.

- SQL — consultas a bancos de dados;
- HTML — descrição de estrutura de páginas;
- Prolog — lógica declarativa.

Esse paradigma é comum em:

- Bancos de dados;
- Linguagens de marcação;
- Sistemas baseados em regras.